

1. Concepts de base

Qu'est-ce qu'une API ?

Une API (Application Programming Interface) est un ensemble de règles et de contrats qui permet à deux applications de communiquer entre elles.

Elle sert à exposer des fonctionnalités ou des données à d'autres programmes, sans avoir besoin de connaître tous les détails de l'implémentation interne.

Qu'est-ce qu'une Web API ?

Une Web API est une API accessible via le web, généralement à travers des requêtes HTTP.

Exemples courants :

- API météo ;
- API de paiement ;
- API de gestion d'utilisateurs ;
- API de notification par e-mail.

Qu'est-ce que HTTP ?

HTTP signifie HyperText Transfer Protocol.

C'est le protocole utilisé pour échanger des informations entre un client et un serveur sur le web.

Les principaux éléments sont :

- une requête envoyée par le client ;
- une réponse renvoyée par le serveur ;
- des méthodes comme GET, POST, PUT, DELETE ;
- des codes de statut HTTP comme 200, 201, 400, 404, 500.

Qu'est-ce qu'une API REST ?

REST (Representational State Transfer) est un style d'architecture pour concevoir des API.

Une API REST repose sur plusieurs principes :

- utilisation des méthodes HTTP ;
- ressources identifiées par des URL ;
- échanges de données généralement en JSON ;
- état de la conversation conservé par le client.

Exemple :

- GET /users → récupérer les utilisateurs ;
- POST /users → créer un utilisateur ;
- PUT /users/1 → modifier un utilisateur ;
- DELETE /users/1 → supprimer un utilisateur.

Concepts associés

- Route : une URL associée à une action spécifique.
 - Endpoint : la ressource exposée par l'API.
 - Requête : ce que le client envoie au serveur.
 - Réponse : ce que le serveur renvoie au client.
 - Payload : les données envoyées dans le body d'une requête.
 - JSON : format de données couramment utilisé dans les API.
-

2. Installation de FastAPI

Pour cette formation, nous allons utiliser FastAPI avec l'option standard.

Installation

Sous Windows, dans un terminal :

```
python -m venv .venv
.venv\Scripts\activate
pip install "fastapi[standard]"
```

Cette installation apporte :

- FastAPI ;
 - Uvicorn (serveur ASGI) ;
 - Pydantic (validation des données) ;
 - outils utiles pour développer rapidement une API.
-

3. Initialisation du projet

Voici une structure simple pour débiter :

```
api/
├── server.py
├── controllers/
├── dto/
├── services/
└── templates/
```

Exemple de fichier principal

Le fichier principal de l'application est généralement un fichier comme :

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

Ensuite, on peut ajouter des routes et organiser le code dans des modules séparés.

Lancer l'application

```
uvicorn server:app --reload
```

L'application sera alors accessible sur :

- <http://127.0.0.1:8000>

4. Concepts abordés dans la première démo

Dans la première démonstration, nous avons vu plusieurs notions essentielles.

4.1 Création d'une application FastAPI

On crée une instance de FastAPI :

```
from fastapi import FastAPI

app = FastAPI()
```

C'est le point d'entrée de l'application.

4.2 Définition des routes

Une route permet d'associer une URL à une fonction Python.

Exemple :

```
@app.get("/hello")
def hello():
    return {"message": "Hello"}
```

4.3 Méthodes HTTP

Nous avons utilisé plusieurs méthodes :

- GET : récupérer une ressource ;
- POST : créer une ressource ;
- PUT : modifier une ressource ;
- DELETE : supprimer une ressource.

4.4 Paramètres

FastAPI permet de récupérer des données de différentes façons :

- Path parameters : dans l'URL ;
- Query parameters : après le ? dans l'URL ;
- Body parameters : dans le corps de la requête.

4.5 Validation des données avec Pydantic

FastAPI utilise Pydantic pour valider les données entrantes.

Cela permet de garantir que les données respectent un schéma défini.

Exemple de concept :

- un champ doit être une chaîne de caractères ;
- un entier doit être strictement positif ;
- un email doit avoir un format valide.

4.6 Les DTO (Data Transfer Object)

Les DTO servent à définir la structure des données échangées.

Ils rendent l'API plus claire et facilitent la validation.

Dans ce projet, on peut voir des fichiers comme :

- dto/hello_request_dto.py
- dto/hello_response_dto.py
- dto/mail_request_dto.py

4.7 Les routers

Pour organiser le code, on peut répartir les routes dans plusieurs fichiers via des routers.

Exemple :

```
from fastapi import APIRouter

router = APIRouter(prefix="/default", tags=["Default"])
```

Cela rend le projet plus propre et plus maintenable.

4.8 Les réponses HTTP

FastAPI permet de définir un code de statut de réponse.

Exemple :

```
@app.post("/mail", status_code=201)
```

Cela permet de renvoyer des réponses appropriées selon l'action réalisée.

4.9 L'injection de dépendances

FastAPI propose un système d'injection de dépendances très pratique.

Cela permet de réutiliser des services ou des objets sans duplication de code.

Dans l'exemple, un service de mail peut être injecté dans une route.

5. Documentation automatique avec Swagger et Redoc

FastAPI génère automatiquement une documentation interactive pour votre API.

Swagger UI

Swagger UI permet de tester les endpoints directement depuis le navigateur.

Une fois votre application lancée, vous pouvez ouvrir :

- <http://127.0.0.1:8000/docs>

Cette page affiche :

- la liste des routes disponibles ;
- les paramètres attendus ;
- les schémas de données ;
- un bouton pour envoyer des requêtes de test.

Redoc

Redoc propose une autre vue de la documentation, souvent plus lisible pour une présentation ou une lecture de référence.

Vous pouvez l'ouvrir ici :

- <http://127.0.0.1:8000/redoc>

Pourquoi c'est utile ?

La documentation automatique permet de :

- comprendre rapidement les endpoints disponibles ;
- tester l'API sans outil externe ;
- partager une documentation toujours à jour ;
- faciliter le travail entre développeurs.